

# Are LLMs Reliable for Quantum Software Development? An Empirical Benchmark Study

Filipe Fernandes

Federal Institute of Southeast of Minas Gerais  
(IF Sudeste MG)

Manhuaçu, Minas Gerais, Brazil  
filipe.fernandes@ifsudestemg.edu.br

## Abstract

Large language models (LLMs) are increasingly being used for code generation, yet their reliability in quantum software development remains largely unexplored. I adapt a one-shot, platform-validated benchmarking methodology to the quantum computing context by evaluating four frontier LLMs (Claude Fable 5, DeepSeek V4 Pro, Gemini 3.1 Pro, and GPT-5.5) on 53 Microsoft Quantum Katas lessons implemented in Q#. The official Katas platform serves as the correctness oracle, and I compare model performance using paired statistical tests. The models exhibit significant differences in their ability to produce correct solutions. Claude Fable 5 solved 94.3% of the lessons, followed by Gemini 3.1 Pro with 90.6% and GPT-5.5 with 86.8%, whereas DeepSeek V4 Pro solved only 58.5%. Moreover, malformed outputs accounted for half of DeepSeek V4 Pro’s failures, indicating limitations in output discipline in addition to correctness. Among the accepted solutions, the primary four-model complete-block analysis found no statistically significant differences for the evaluated quantum-efficiency metrics, including gate count, qubit usage, measurement count, and a proxy for circuit depth. However, source code length differed significantly across models, with Claude Fable 5 producing the fewest source lines and GPT-5.5 the most. These findings indicate that model selection primarily affects correctness and adherence to the expected output format, while having limited influence on the quantum-resource efficiency of functionally correct solutions.

## Keywords

Large Language Models, Quantum Software Engineering, Q#, Code Generation, Empirical Study, Quantum Katas

## 1 Introduction

LLMs are routine coding assistants extensively evaluated for classical code [4, 18]. Quantum studies focus on Qiskit, fine-tuned models, or developer-support tasks [3, 6, 8], leaving frontier general-purpose models underexplored. Here, *reliable* has a bounded, operational meaning, i.e., whether a model’s first response yields Q# code accepted by the official tests for a small, self-contained Kata. I do not evaluate larger applications, debugging, maintenance, integration, repeated executions, or long-term reliability; claims are limited to one-shot task-level correctness in this benchmark.

To address this gap, I evaluate the reliability of four frontier LLMs—Claude Fable 5, DeepSeek V4 Pro, Gemini 3.1 Pro, and GPT-5.5—in generating Q# solutions for 54 Microsoft Quantum Katas lessons stratified by difficulty (53 valid) under a one-shot, platform-validated protocol. With paired non-parametric tests, I compare

their (RQ1) *task success rate*, the proportion of first responses accepted by the official platform; (RQ2) the *quantum-resource efficiency* of accepted solutions; and (RQ3) their classical *software structure*. The task success rate ranges from 94.3% (Claude Fable 5) to 58.5% (DeepSeek V4 Pro), and only DeepSeek differs significantly from the other models; the primary complete-block analysis finds no significant quantum-resource differences but they differ significantly in source-code size.

This work offers: (i) a platform-validated, statistically paired comparison of four frontier LLM families on 53 Q# programming tasks; (ii) a documented, reproducible suite of quantum-resource and software-structure metrics for Q#; and (iii) a complete replication package.

## 2 Background and Related Work

**Quantum software engineering and Q#.** Q# [20] is a typed, quantum-native language: programs are organized into *operations* whose signatures declare Adjoint/Controlled functor support, qubits are allocated in scoped use/borrow statements, and *within/apply* expresses the compute-act-uncompute pattern—constructs that ground the metrics of Section 3.5.3. The Microsoft Quantum Katas [17], the official collection of Q# exercises with executable unit tests, serve here as both task source and correctness oracle (Section 3.2).

**Evaluating LLM-generated classical code.** Execution-based benchmarks such as HumanEval established unit tests as the oracle for LLM-generated code [4], later extended to further lifecycle stages [15] and cataloged by a survey of 291 software-engineering benchmarks [4]. Quality-focused studies compared GPT-based assessments with SonarQube over 1,641 Java classes [19] and evaluated 2,464 LLM solutions for 616 Python LeetCode problems [1]; all target classical languages.

**The methodological baseline.** Oliveira et al. [18] compared six LLMs on 204 Java LeetCode problems with a fixed prompt, one response per model-task pair, LeetCode acceptance as oracle, SonarQube metrics over accepted solutions, and Shapiro-Wilk plus Mann-Whitney U tests; all null hypotheses were rejected, with Gemini 2.5 Pro reaching the highest task success rate (82.35%). I preserved these defining elements and adapted them to the quantum setting: the Katas replace LeetCode as task source and oracle, verifying quantum states and unitaries rather than input-output pairs; a documented, regex-based suite of quantum-resource and software-structure metrics replaces SonarQube, which does not analyze Q#; and paired statistics (Cochran’s Q/McNemar, Friedman/Wilcoxon, Holm-corrected) replace independent-samples tests, since every model attempts the same lessons.

**LLMs and quantum software.** Work connecting LLMs to quantum code concentrates on the Qiskit/Python ecosystem: the Qiskit Code Assistant fine-tunes LLMs for quantum code [8], Qiskit HumanEval benchmarks such models with execution-based tests [22], and Campbell et al. [3] improve generation accuracy with a multi-agent, error-correction-aware pipeline. Other studies assess LLM support for explaining quantum algorithms [6] or answering quantum software questions [14], while hybrid quantum-AI optimization [11] and quantum-annealing-based test case minimization [24] explore the reverse synergy, without evaluating generated quantum code.

In summary, prior evaluations are either thorough but classical, or quantum but Qiskit-centric, accuracy-oriented, and without statistical comparison across frontier models. To the best of my knowledge, no previous study offers a platform-validated, statistically paired comparison of frontier general-purpose LLMs generating Q# code that jointly examines correctness, quantum-resource usage, and software structure—the evidence my research questions provide (Section 3).

### 3 Study Design

I adopt a quantitative, one-shot, platform-validated design: tasks from the Microsoft Quantum Katas [17], solutions in Q# [20], and the official Katas platform as correctness oracle. I version all experimental artifacts and make them available in the replication package. This section presents the goal and research questions, the tasks, the evaluated LLMs, the prompting protocol, the metrics, and the statistical analysis.

#### 3.1 Goal and Research Questions

The goal of this study is to evaluate whether frontier LLMs can be relied upon to generate quantum code. I structure this goal using the Goal–Question–Metric (GQM) approach [2]: I *analyze* Q# solutions generated by frontier LLMs *for the purpose of* evaluating their reliability *with respect to* correctness, quantum-resource efficiency, and classical code structure, *from the point of view of* quantum software developers and researchers, *in the context of* the Microsoft Quantum Katas. Each facet is refined into one research question, operationalized by the attributes defined in Section 3.5:

- **RQ1:** Is there a statistically significant difference in task success rate, defined as the proportion of tasks whose generated solutions are accepted by the Katas platform, among the evaluated LLMs?
- **RQ2:** Among accepted solutions, is there a statistically significant difference in quantum-resource efficiency among the evaluated LLMs?
- **RQ3:** Among accepted solutions, is there a statistically significant difference in classical software-structure metrics among the evaluated LLMs?

For each research question,  $H_0$  states that the named attribute does not differ among the evaluated LLMs, and  $H_1$  that at least one LLM differs. RQ1 examines the primary condition for reliability: functional correctness according to the official platform. RQ2 asks whether correct solutions differ in quantum resources (gates, qubits, measurements, resets, circuit depth), which affect execution cost on noisy hardware. RQ3 asks whether they differ in structural properties (size, operation calls, control flow, ancilla allocation,

within/apply), which describe code size and structure but do not directly measure readability or maintainability.

#### 3.2 Experimental Objects

The Microsoft Quantum Katas [17] are an official, self-contained collection of Q# programming exercises. Each lesson provides a problem statement and a Q# operation signature to be completed, and the online platform compiles the submitted code and runs the lesson’s unit tests, returning an explicit verdict. This combination of fixed task statements, starter code, and an executable oracle makes the Katas a quantum analogue of online judges such as LeetCode.

I first extracted the 201 lessons available on the official Katas website via web scraping. From this pool I selected 54 lessons, stratified into 18 *basic*, 18 *intermediate*, and 18 *advanced* lessons according to a documented difficulty-inference rule, since the Katas provide no official difficulty labels. The selected lessons span 20 chapters, from foundational gate and measurement topics to complete algorithms and protocols (e.g., Grover’s search, the quantum Fourier transform, and quantum error correction).

I excluded one lesson (044) for a reason unrelated to the LLMs: the Katas website provides an invalid link for it, which I verified manually, so its submissions cannot be evaluated. The final dataset includes 53 lessons, i.e.,  $53 \times 4 = 212$  model–task pairs.

#### 3.3 Evaluated LLMs

I evaluated four frontier models, one recent, API-accessible representative of each of four leading LLM families: *Claude Fable 5* (Anthropic), *DeepSeek V4 Pro* (DeepSeek), *Gemini 3.1 Pro* (Google), and *GPT-5.5* (OpenAI). I shortlisted the families from the Vellum AI LLM leaderboard [21] using the *Best Overall* benchmark (Humanity’s Last Exam) as the initial criterion. Selecting one representative per family avoids concentrating the comparison on a single provider and lets distinct architectures be compared under the same protocol. I accessed all models through the OpenRouter<sup>1</sup> API with non-streaming requests and temperature 0, and I logged for every request the model identifier, usage, latency, and cost.

I omitted top\_p, maximum output tokens, seed, reasoning, tools, and provider route; OpenRouter defaults and automatic routing applied, and no seed was returned. Observed routes were Amazon Bedrock/Anthropic, 12 DeepSeek providers, Google/Google AI Studio, and Azure/OpenAI. Generation ran from 2026-07-07 21:16 to 2026-07-08 00:40 UTC. The artifact’s provenance.csv records exact requested/returned model IDs, timestamps, and per-call routes.

Table 1 reports generation overhead for the 53 valid lessons; the 212 requests cost US\$4.0708 in total.

#### 3.4 Prompting Protocol

I used one fixed system–user prompt for every model, instantiated only with the lesson statement and Q# starter code; its full text is in the replication package. Before the experiment, I validated the template with Gemini 2.5 Flash—a model outside the evaluated set—through the Google AI Studio API<sup>2</sup>. I generated exactly *one response per model–task pair*, kept the *first response*, and applied no feedback, retries, repair prompts, or manual improvements. Before validation,

<sup>1</sup><https://openrouter.ai/>

<sup>2</sup><https://aistudio.google.com/>

**Table 1: Generation overhead per model. Paired values are mean/median per request; costs are in US dollars.**

| Model           | Input tok.    | Output tok. | Latency (s)   |
|-----------------|---------------|-------------|---------------|
| Claude Fable 5  | 581/535       | 280/128     | 9.44/5.97     |
| DeepSeek V4 Pro | 355/332       | 1770/656    | 36.94/12.84   |
| Gemini 3.1 Pro  | 363/338       | 2163/1312   | 18.29/13.68   |
| GPT-5.5         | 376/347       | 891/591     | 19.91/13.27   |
| Model           | Cost/request  | Total cost  | Cost/accepted |
| Claude Fable 5  | .01982/.01202 | 1.05038     | .02101        |
| DeepSeek V4 Pro | .00169/.00072 | 0.08982     | .00290        |
| Gemini 3.1 Pro  | .02669/.01665 | 1.41442     | .02947        |
| GPT-5.5         | .02861/.01929 | 1.51619     | .03296        |

I only mechanically extracted code from Markdown fences. I never included reference solutions, hints, or unit-test implementations in prompts, and I logged every raw prompt, raw response, and extracted program.

### 3.5 Correctness and Quality Metrics

RQ1 uses the acceptance verdict of the official Katas platform, whereas RQ2 and RQ3 use quantum-resource and software-structure metrics extracted from the *accepted* solutions. Because SonarQube-style analyzers are not available for Q#, I extract these metrics with a documented, regex-based static pipeline whose definitions I froze before the analysis and applied identically to every model. I strip comments and strings before matching and validate the extractor with built-in sanity checks.

**3.5.1 Correctness and Task Success Rate (RQ1).** I submitted each of the 216 generated solutions to the official online Microsoft Quantum Katas platform, which compiles the code and executes the lesson’s unit tests. I recorded the platform verdict and message with timestamps. A solution is *accepted* if and only if the platform reports success (compiled and passed all tests, no manual repair). Task success rate is the fraction of valid lessons whose solution is accepted. I further classified rejected solutions from the platform messages into four failure categories: *malformed output* (the response was not extractable, compilable Q# code), *compile error* (subdivided into syntax and semantic/type errors), *runtime error*, and *failed tests*.

I store all experimental records in a single JSON structure that keeps, per lesson, the task metadata and one record per model with the generated code, usage, latency, raw response, and the appended platform verdict, message, and timestamp, so every acceptance decision is traceable to its originating request; I document the schema in the replication package.

**3.5.2 Quantum Metrics (RQ2).** *Gate count* is the number of mentions of a fixed gate vocabulary comprising Pauli gates Clifford-related gates (H, S, T), rotations, multi-qubit gates (CNOT, CCNOT, SWAP, CZ, CY), measurements, and resets; mentions cover direct calls, functor applications and references passed as arguments. *Gate count by type* records the same information per vocabulary entry. The *circuit-depth proxy* is defined as *circuit\_depth\_approx* = *gate\_count\_total*; it is explicitly a syntactic approximation that assumes serial execution and ignores gate parallelism, loop trip counts, array-wide applications, and the decomposition of library operations, serving as an ordering proxy rather than a physical

**Table 2: Task success rate per LLM over the 53 valid lessons, with Wilson 95% confidence intervals.**

| Model           | Accepted | Task success rate | 95% CI         |
|-----------------|----------|-------------------|----------------|
| Claude Fable 5  | 50/53    | 0.943             | [0.846, 0.981] |
| Gemini 3.1 Pro  | 48/53    | 0.906             | [0.797, 0.959] |
| GPT-5.5         | 46/53    | 0.868             | [0.752, 0.935] |
| DeepSeek V4 Pro | 31/53    | 0.585             | [0.451, 0.707] |

depth. *Qubit count* sums scalar Qubit parameters, `Qubit[]` parameters (counted as one, since their size is not statically known), and qubits allocated by `use/borrow` statements. I also record the *CNOT*, *controlled-functor*, *multi-qubit gate*, *measurement*, and *reset counts*.

**3.5.3 Software Metrics (RQ3).** *Source LOC* counts non-blank, non-comment lines. *Operation call count* counts call sites after removing declarations and language keywords, plus functor references that are not directly invoked, and *distinct operation types* counts the number of distinct operation or function names among them. *Classical control count* totals the occurrences of the keywords `if`, `elif`, `else`, `for`, `while`, `repeat`, `until`, `return`, `fail`, `within`, and `apply`. *Ancilla allocation count* counts `use/borrow` allocation statements. I also record the number of `within` / `apply` blocks and whether the conjugation pattern is used.

### 3.6 Statistical Analysis

My design is *paired*: every model attempts the same lessons, so I adopt paired procedures, following recommendations for comparing multiple algorithms over the same tasks [7]. For the binary RQ1 outcome I apply Cochran’s Q test [5] as the omnibus procedure over the 53 valid lessons, followed by pairwise exact McNemar tests [16] with Holm correction [10] for the six model pairs. For the numeric RQ2/RQ3 outcomes I use the Friedman test [9] as the omnibus procedure, reporting Kendall’s W [12] as effect size, followed by pairwise two-sided Wilcoxon signed-rank tests [23] with Holm correction applied within each metric family and the matched-pairs rank-biserial correlation [13] as effect size. I compute numeric tests over the 31 *complete blocks* (lessons accepted by all four models). All tests use  $\alpha = 0.05$ , and I report exact *p*-values. To assess complete-case selection, I also repeat every pairwise Wilcoxon test on all lessons accepted by both models in that pair (31–46 shared lessons), retaining Holm correction across the six pairs per metric.

## 4 Results

This section reports the results for each research question; all values were produced by the analysis notebooks of the replication package.

### 4.1 RQ1: Task Success Rate

*Task success rate* (Section 3.1) is the proportion of the 53 valid lessons whose solution was accepted by the platform (Section 3.5).

Table 2 summarizes the task success rate of each model. Claude Fable 5 achieved the highest rate (50/53, 94.3%), followed by Gemini 3.1 Pro (48/53, 90.6%) and GPT-5.5 (46/53, 86.8%), whereas DeepSeek V4 Pro solved substantially fewer lessons (31/53, 58.5%). Lesson-level agreement was high at the top of the ranking: 31 of the 53

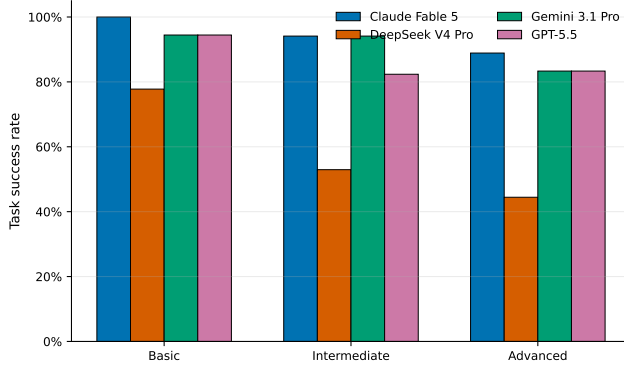


Figure 1: Task success rate per LLM stratified by lesson difficulty.

Table 3: Concrete output excerpts and recorded Katas verdicts.

| Outcome       | Output excerpt and verdict                                                                         |
|---------------|----------------------------------------------------------------------------------------------------|
| Successful    | Claude, lesson 001: $X(q)$ ; – “Correct solution.”                                                 |
| Malformed     | DeepSeek, lesson 007: “‘qsharp ... ‘ – “unrecognized character.”                                   |
| Compile error | GPT-5.5, lesson 021: <code>for (q in qs) { ... }</code> – “expected ‘)’”, found <code>in</code> .” |
| Failed tests  | GPT-5.5, lesson 010: $H(qs[0])$ ; $Z(qs[0])$ ; – “Solution is incorrect.”                          |

lessons were solved by all four models, 12 by exactly three, 5 by exactly two, and 5 by exactly one, and no lesson was left unsolved by all models.

Figure 1 stratifies task success rate by difficulty. All models degrade as difficulty increases, but the gradient is much steeper for DeepSeek V4 Pro (77.8% on basic, 52.9% on intermediate, and 44.4% on advanced lessons) than for Claude Fable 5 (100%, 94.1%, and 88.9%, respectively), Gemini 3.1 Pro (94.4%, 94.1%, 83.3%), and GPT-5.5 (94.4%, 82.4%, 83.3%).

The nature of the failures is as informative as their number (Figure 2). Claude Fable 5 failed only 3 lessons (one semantic compile error, one runtime error, one failed test), Gemini 3.1 Pro failed 5, and GPT-5.5 failed 7, with GPT-5.5’s failures dominated by syntax-level compile errors (4 of 7). In contrast, half of DeepSeek V4 Pro’s 22 failures (11) were *malformed outputs*, i.e., responses from which no compilable Q# solution could be mechanically extracted, in addition to 7 compile errors and 4 test failures.

Table 3 gives logged examples; full programs and verdicts are in the replication package.

Cochran’s  $Q$  test (Section 3.6) rejects the null hypothesis of equal task success rates among the four models ( $Q = 37.99$ ,  $df = 3$ ,  $p = 2.85 \times 10^{-8}$ ). Table 4 reports the pairwise exact McNemar tests with Holm correction. DeepSeek V4 Pro differs significantly from each of the other three models: it never solved a lesson that Claude Fable 5 missed, while Claude solved 19 lessons that DeepSeek missed ( $p_{\text{Holm}} = 2.3 \times 10^{-5}$ ); the corresponding discordant counts against Gemini 3.1 Pro and GPT-5.5 are 17 versus 0 ( $p_{\text{Holm}} = 7.6 \times 10^{-5}$ ) and 15 versus 0 ( $p_{\text{Holm}} = 2.4 \times 10^{-4}$ ). No significant differences were detected among Claude Fable 5, Gemini 3.1 Pro, and GPT-5.5 ( $p_{\text{Holm}} = 1.0$  for all three pairs).

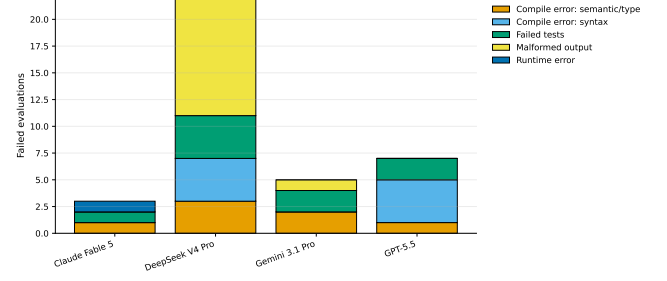


Figure 2: Failure classes per LLM over the rejected solutions.

Table 4: Pairwise exact McNemar tests with Holm correction for RQ1. Discordant counts give the lessons solved only by model A and only by model B.

| Model A        | Model B         | Only A/<br>Only B | $p_{\text{Holm}}$    | Sig. |
|----------------|-----------------|-------------------|----------------------|------|
| Claude Fable 5 | DeepSeek V4 Pro | 19 / 0            | $2.3 \times 10^{-5}$ | yes  |
| Gemini 3.1 Pro | DeepSeek V4 Pro | 17 / 0            | $7.6 \times 10^{-5}$ | yes  |
| GPT-5.5        | DeepSeek V4 Pro | 15 / 0            | $2.4 \times 10^{-4}$ | yes  |
| Claude Fable 5 | Gemini 3.1 Pro  | 4 / 2             | 1.000                | no   |
| Claude Fable 5 | GPT-5.5         | 7 / 3             | 1.000                | no   |
| Gemini 3.1 Pro | GPT-5.5         | 3 / 1             | 1.000                | no   |

Table 5: Friedman tests for the quantum-efficiency metrics (RQ2) over the 31 lessons accepted by all models.

| Metric                 | $\chi^2$                   | $p$ -value | Kendall’s $W$ |
|------------------------|----------------------------|------------|---------------|
| Total gate count       | 4.56                       | 0.207      | 0.049         |
| Circuit depth (proxy)  | 4.56                       | 0.207      | 0.049         |
| Qubit count            | 3.00                       | 0.392      | 0.032         |
| CNOT count             | 4.71                       | 0.194      | 0.051         |
| Controlled count       | 6.00                       | 0.112      | 0.065         |
| Multi-qubit gate count | 7.20                       | 0.066      | 0.077         |
| Measurement count      | no variation across models |            |               |
| Reset count            | 7.25                       | 0.064      | 0.078         |

**Answer to RQ1.** There is a statistically significant difference in task success rate among the evaluated LLMs. The difference is driven by DeepSeek V4 Pro, whose rate is significantly lower than Claude Fable 5, Gemini 3.1 Pro, and GPT-5.5; no pairwise difference was detected among the latter three.

## 4.2 RQ2: Quantum Efficiency

*Quantum efficiency* (Section 3.1) concerns the quantum resources consumed by the *accepted* solutions, measured by the metrics of Section 3.5.2. Table 5 summarizes the Friedman tests for the quantum metrics, computed over the 31 complete blocks. No primary test is significant; the closest are reset ( $p = .064$ ) and multi-qubit gate count ( $p = .066$ ), measurement count has no variation, and all effect sizes are negligible to small ( $W \leq .078$ ).

Descriptively, the mean total gate count per accepted solution ranges narrowly from 2.04 (Claude Fable 5) to 2.63 (GPT-5.5), with

**Table 6: Friedman tests for the software-structure metrics (RQ3) over the 31 lessons accepted by all models.**

| Metric                   | $\chi^2$ | $p$ -value            | Kendall's W |
|--------------------------|----------|-----------------------|-------------|
| Source LOC               | 35.88    | $7.95 \times 10^{-8}$ | 0.386       |
| Classical control count  | 10.56    | 0.014                 | 0.114       |
| Operation call count     | 4.97     | 0.174                 | 0.053       |
| Distinct operation types | 1.34     | 0.719                 | 0.014       |
| Ancilla allocation count | 3.00     | 0.392                 | 0.032       |
| within block count       | 3.00     | 0.392                 | 0.032       |
| apply block count        | 3.00     | 0.392                 | 0.032       |

a common median of 2 gates (1.5 for Claude), and the mean qubit count ranges from 1.34 to 1.48 with a common median of 1. The gate-type profile is likewise homogeneous: all models rely predominantly on the same small set of operations—H, X, CNOT, and measurements—with similar per-solution means.

The pairwise shared-task sensitivity analysis uses 31–46 lessons per pair. It finds one RQ2 contrast: Claude has fewer gate mentions than GPT-5.5 on their 43 jointly accepted lessons ( $p_{\text{Holm}} = .0199$ ,  $r = -.936$ ). The identical result for the depth proxy follows from its definition as total gate count; no other quantum comparison is significant.

**Answer to RQ2.** Primary complete-block tests detect no significant quantum-metric difference; a Claude–GPT-5.5 gate-count sensitivity contrast limits this finding to the four-model subset.

### 4.3 RQ3: Software Structure

Finally, *software structure* captures the classical dimension of the accepted solutions, measured by the metrics of Section 3.5.3. Table 6 summarizes the Friedman tests for the software-structure metrics. Source LOC differs significantly among the models ( $\chi^2 = 35.88$ ,  $p = 7.95 \times 10^{-8}$ ,  $W = 0.386$ , a moderate effect). I interpret LOC only as code size or verbosity, not as readability or maintainability. The classical control count is also significant at the omnibus level ( $\chi^2 = 10.56$ ,  $p = 0.014$ ,  $W = 0.114$ ), whereas the operation call count ( $p = 0.174$ ), distinct operation types ( $p = 0.719$ ), ancilla allocation count ( $p = 0.392$ ), and within/apply counts ( $p = 0.392$ ) show no significant differences.

Across accepted solutions, Claude Fable 5 produced the fewest source lines (mean 8.76 LOC, median 7), followed by DeepSeek V4 Pro (mean 9.97, median 6), Gemini 3.1 Pro (mean 10.63, median 8), and GPT-5.5 the most (mean 12.11, median 8.5). Table 7 reports the pairwise Wilcoxon signed-rank tests for LOC. Four pairs are significant after Holm correction: Claude Fable 5 writes significantly shorter solutions than DeepSeek V4 Pro ( $p_{\text{Holm}} = 6.1 \times 10^{-4}$ ,  $r = -0.93$ ), Gemini 3.1 Pro ( $p_{\text{Holm}} = 2.9 \times 10^{-3}$ ,  $r = -0.95$ ), and GPT-5.5 ( $p_{\text{Holm}} = 9.1 \times 10^{-5}$ ,  $r = -1.00$ , i.e., GPT-5.5 was longer on every complete-block lesson where they differed), and Gemini 3.1 Pro writes significantly shorter solutions than GPT-5.5 ( $p_{\text{Holm}} = 1.1 \times 10^{-2}$ ,  $r = -0.90$ ). The DeepSeek–Gemini and DeepSeek–GPT-5.5 pairs are not significant. The sensitivity analysis preserves these four LOC differences and also finds fewer operation calls for Claude than GPT-5.5 on their 43 shared accepted lessons ( $p_{\text{Holm}} = .0199$ ,  $r = -.936$ ); no other additional software-structure pair is significant.

**Table 7: Pairwise Wilcoxon signed-rank tests with Holm correction for source LOC (RQ3), with matched-pairs rank-biserial correlation  $r$ . Negative  $r$  indicates shorter solutions for model A.**

| Model A         | Model B         | $p_{\text{Holm}}$    | $r$   | Sig. |
|-----------------|-----------------|----------------------|-------|------|
| Claude Fable 5  | GPT-5.5         | $9.1 \times 10^{-5}$ | −1.00 | yes  |
| Claude Fable 5  | DeepSeek V4 Pro | $6.1 \times 10^{-4}$ | −0.93 | yes  |
| Claude Fable 5  | Gemini 3.1 Pro  | $2.9 \times 10^{-3}$ | −0.95 | yes  |
| Gemini 3.1 Pro  | GPT-5.5         | $1.1 \times 10^{-2}$ | −0.90 | yes  |
| DeepSeek V4 Pro | GPT-5.5         | 0.082                | −0.66 | no   |
| DeepSeek V4 Pro | Gemini 3.1 Pro  | 0.162                | +0.47 | no   |

For the classical control count, despite the significant omnibus result, no pairwise comparison survived the Holm correction; the smallest adjusted  $p$ -value was 0.087 (Claude Fable 5 versus GPT-5.5), consistent with GPT-5.5’s higher mean (2.15 versus 1.64 control-flow keywords per accepted solution). The within/apply conjugation pattern was rarely used by any model: it appears in 6.0% of Claude Fable 5’s accepted solutions (3/50), 6.3% of Gemini 3.1 Pro’s (3/48), 4.3% of GPT-5.5’s (2/46), and 3.2% of DeepSeek V4 Pro’s (1/31).

**Answer to RQ3.** Source LOC differs significantly among the LLMs: Claude Fable 5 produces the fewest source lines and GPT-5.5 the most. Complete-block tests detect no other pairwise difference; sensitivity analysis additionally separates Claude and GPT-5.5 on operation calls.

## 5 Discussion

The results answer the three research questions with a clear overall pattern: the choice of LLM strongly affects *whether* a quantum task is solved (RQ1) and *how verbosely* it is coded (RQ3), whereas the primary analysis detects no quantum-resource difference (RQ2).

### 5.1 Reliability of Frontier LLMs for Quantum Programming

Three of the four models solved the large majority of the lessons in a strict one-shot setting. For such well-specified, kata-sized tasks, frontier LLMs are already usable as first-draft generators of Q# code. Pairwise tests do not resolve differences among Claude Fable 5, Gemini 3.1 Pro, and GPT-5.5; this is absence of evidence, not evidence of equal success rates.

DeepSeek V4 Pro is the exception, and the nature of its failures is informative: half of its rejections were malformed outputs rather than incorrect quantum logic. Because format deviations are unrecoverable one-shot, part of DeepSeek’s gap reflects instruction-following rather than quantum reasoning. Retries or repair prompts might narrow this gap but were outside the protocol. The difficulty stratification reinforces this reading: DeepSeek’s task success rate drops sharply from basic to advanced lessons, while the other models degrade more gradually.

## 5.2 Quantum Resource Efficiency

The primary complete-block results for quantum-efficiency metrics should be interpreted in light of the task profile. Katas lessons are small, tightly specified exercises whose unit tests constrain the target state or unitary; once a solution is correct, there is limited room for resource variation, as the low medians and negligible effect sizes reflect. The correctness constraints compress the space of acceptable implementations, so accepted solutions often show similar gate profiles. I refrain from claiming that frontier LLMs are equally resource-efficient in general; the supported observation is that, *conditioned on correctness for kata-sized tasks*, their quantum-resource usage shows no significant four-model difference. However, the pairwise sensitivity analysis finds fewer gate mentions for Claude than GPT-5.5, so the null result is not robust to every shared-task subset. Whether this persists on larger programs with real optimization headroom remains open.

## 5.3 Classical Software Structure

Primary complete-block tests detect no quantum-resource difference, whereas source LOC does differ: Claude Fable 5 has the fewest lines and GPT-5.5 the most, with very large paired effects. LOC measures size or verbosity rather than quality. Short code can be dense or contain atoms of confusion, whereas longer code can use explanatory variables or scaffolding. Because I performed no human comprehension or maintainability assessment, these results establish no readability advantage for any model. The rarity of *within/apply* (at most 6.3% of accepted solutions) suggests idiomatic Q# constructs are underused in favor of explicit gate sequences.

## 5.4 Main Contributions

The clearest differences are task success and code structure; the isolated gate-count sensitivity result shows that resource studies need tasks with optimization headroom. Frontier LLMs can generate small Q# routines one shot, but model choice affects correctness and output discipline, idiom use needs review, and LOC measures size rather than quality. Malformed output should be reported separately from semantic failure. A fixed prompt yielded above 86% task success for three families, though format instability remains model-dependent.

## 6 Threats to Validity

**Construct validity.** Task success rate operationalizes correctness as acceptance by the Katas platform, whose unit tests are authoritative but do not prove full semantic equivalence, a residual risk. One-shot acceptance also folds output-format compliance into the construct, isolated as the *malformed output* failure class (Section 3.5.1). The RQ2/RQ3 metrics are regex-based syntactic approximations without AST support, and the circuit-depth proxy equals the total gate count, ignoring parallelism (Section 3.5.2); the definitions I froze and applied identically to every model keep comparisons fair, but absolute values are not physical estimates. I inferred the difficulty labels by a documented rule; misclassification would distort RQ1's stratified view, not the overall tests.

**Internal validity.** The online Katas platform is a moving oracle, and the excluded lesson\_044 shows that platform-side failures

occur (Section 3.2); I logged every verdict, message, and timestamp and applied the exclusion rule before any analysis. I used one fixed prompt for all models after I validated it with a model outside the evaluated set (Section 3.4), so differences may partly reflect prompt-model fit. I queried the models under their providers' default settings through OpenRouter; I treat each model *as served*. Restricting RQ2/RQ3 to the 31 complete blocks is required by the paired tests but skews them toward easier lessons. The pairwise sensitivity check expands this to 31–46 shared accepted lessons but remains conditioned on joint success; neither analysis represents all accepted solutions, and each pair can have a different task mix.

**External validity.** The strongest limitation is the exclusive use of Q#, a standalone, typed quantum language, whereas Qiskit and PennyLane are Python-embedded SDKs with far more public code [8, 22]. LLMs have plausibly seen much less Q# during training, and uneven familiarity can affect generation, compilation, and format compliance. The shared language, katas, and oracle keep the comparison internally fair, but my conclusions are delimited to Q# and require replication elsewhere. The tasks are kata-sized, smaller than industrial quantum programs, and the study covers four model versions from a single generation window. Training-data contamination with public Katas solutions cannot be ruled out—prompts never included reference solutions, hints, or tests—and could inflate absolute success rates, possibly unevenly across models.

**Conclusion validity.** I collected one response per model-task pair, so LLM nondeterminism is not averaged out. The paired workflow uses exact *p*-values and Holm correction within, but not across, metric families. With 53 lessons (31 complete blocks), power is limited: some RQ2 metrics approached the threshold ( $p = 0.064 - 0.066$ ), so non-significant results indicate absence of evidence with negligible-to-small effects (Kendall's  $W \leq 0.078$ ), not equivalence.

## 7 Concluding Remarks

I compared four frontier LLMs on 53 Q# Katas under a one-shot, platform-validated protocol. Task success differs significantly, driven by DeepSeek V4 Pro (RQ1); the complete-block analysis detects no quantum-resource difference, although one pairwise sensitivity contrast emerges (RQ2); and source LOC differs, with Claude Fable 5 having the fewest lines and GPT-5.5 the most (RQ3).

I contribute a platform-validated, one-shot quantum-code benchmark method, a reproducible Q# metric suite, an empirical comparison, and a replication package.

These findings are limited to kata-sized Q# tasks under a one-shot protocol. Future work should use larger tasks, repeated samples, AST-based metrics, and simulator or hardware resource estimates.

## Artifact Availability

I release the dataset, prompts and responses, logs and verdicts, metrics, and notebooks at <https://github.com/filipefernandesphd/wqse26-llms-oneshot-qsharp-gen>.

## Acknowledgements

I used ChatGPT and Claude for API code, analysis, figures, and language editing; I reviewed, validated, and corrected all AI-assisted outputs.

## References

- [1] Bernardo Aquino, Aline Brito, Cleiton Tavares, Danilo Boechat, and João Batisteli. 2025. Analisando a Qualidade e Eficácia de Códigos Gerados por LLMs: Um Estudo com Problemas da Plataforma LeetCode. In *Anais do XIII Workshop de Visualização, Evolução e Manutenção de Software* (Recife/PE). SBC, Porto Alegre, RS, Brasil, 25–36. doi:10.5753/vem.2025.14303
- [2] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. 1994. The Goal Question Metric Approach. In *Encyclopedia of Software Engineering*. John Wiley & Sons, New York, NY, USA, 528–532.
- [3] Charlie Campbell, Hao Mark Chen, Wayne Luk, and Hongxiang Fan. 2025. Enhancing LLM-based Quantum Code Generation with Multi-Agent Optimization and Quantum Error Correction. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. 1–7. doi:10.1109/DAC63849.2025.11133316
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG]. <https://arxiv.org/abs/2107.03374>
- [5] W. G. Cochran. 1950. The Comparison of Percentages in Matched Samples. *Biometrika* 37, 3/4 (1950), 256–266. <http://www.jstor.org/stable/2332378>
- [6] Giordano d'Aloisio, Sophie Fortz, Carol Hanna, Daniel Fortunato, Avner Ben-soussan, Eñaut Mendiluze Usandizaga, and Federica Sarro. 2024. Exploring LLM-Driven Explanations for Quantum Algorithms. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (Barcelona, Spain) (ESEM '24). Association for Computing Machinery, New York, NY, USA, 475–481. doi:10.1145/3674805.3690753
- [7] Janez Demšar. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *The Journal of Machine Learning Research* 7 (Dec. 2006), 1–30.
- [8] Nicolas Dupuis, Luca Buratti, Sanjay Vishwakarma, Aitana Viudes Forrat, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit Code Assistant: Training LLMs for generating Quantum Computing Code. In *2024 IEEE LLM Aided Design Workshop (LAD)*. 1–4. doi:10.1109/LAD62341.2024.10691762
- [9] Milton Friedman. 1937. The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance. *Journal of the American Statistical Association* 32, 200 (1937), 675–701. doi:10.1080/01621459.1937.10503522
- [10] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. <http://www.jstor.org/stable/4615733>
- [11] Anil Kumar Jonnalagadda, Sudeep Acharya, Venkata Reddy Pasam, and Rohan Shahane. 2025. Quantum-AI Hybrid Algorithms for Solving Large-Scale Engineering Problems Using LLM-Driven Optimizations. In *2025 5th International Conference on Intelligent Technologies (CONIT)*. 1–8. doi:10.1109/CONIT65521.2025.11166726
- [12] M. G. Kendall and B. Babington Smith. 1939. The Problem of m Rankings. *The Annals of Mathematical Statistics* 10, 3 (1939), 275–287. <http://www.jstor.org/stable/2235668>
- [13] Dave S. Kerby. 2014. The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation. *Comprehensive Psychology* 3 (2014), 11.IT.3.1.
- [14] Nek Dil Khan, Safia Kanwal, Javed Ali Khan, Arif Ali Khan, and Muhammad Azeem Akbar. 2026. Measuring the Effectiveness of Large Language Model for Answer Generation: A Case of Quantum Software Engineering. In *Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26)*. Association for Computing Machinery, New York, NY, USA, 1343–1350. doi:10.1145/3748522.3779916
- [15] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2025. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. In *Proceedings of the 31st International Conference on Computational Linguistics*. Association for Computational Linguistics, Abu Dhabi, UAE, 7511–7531. doi:10.1145/3748522.377991
- [16] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996
- [17] Microsoft. 2026. The Quantum Katas. <https://quantum.microsoft.com/en-us/tools/quantum-katas>. Accessed: 2026-06-30.
- [18] Marco Oliveira, Pedro Silveira, and Michelle Andrade. 2025. Avaliação de Qualidade de Código Java gerado por Large Language Models. In *Anais da IX Escola Regional de Engenharia de Software* (Chapeco/SC). SBC, Porto Alegre, RS, Brasil, 70–79. doi:10.5753/eres.2025.16845
- [19] Igor Regis da Silva Simões and Elaine Venson. 2024. Evaluating Source Code Quality with Large Language Models: a comparative study. In *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS '24)*. Association for Computing Machinery, New York, NY, USA, 103–113. doi:10.1145/3701625.3701650
- [20] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018* (Vienna, Austria) (RWDSL2018). Association for Computing Machinery, New York, NY, USA, Article 7, 10 pages. doi:10.1145/3183895.3183901
- [21] Vellum AI. 2026. LLM Leaderboard. <https://www.vellum.ai/llm-leaderboard>. Accessed: 2026-06-30.
- [22] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit HumanEval: An Evaluation Benchmark for Quantum Code Generative Models. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 01. 1169–1176. doi:10.1109/QCE60285.2024.00137
- [23] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. <http://www.jstor.org/stable/3001968>
- [24] Huixiang Zhang and Mahzabeen Emu. 2025. Quantum-Guided Test Case Minimization for LLM-Based Code Generation. In *2025 IEEE International Conference on Collaborative Advances in Software and Computing (CASCON)*. 609–610. doi:10.1109/CASCON66301.2025.00108